RIVERTY · LEGAL TECHNOLOGY · AI CONTRACT INTELLIGENCE

Contract Review Intelligence System

AI-powered search, comparison, and compliance checking — built for your legal team, ready for your Azure stack.

RAG Architecture | Hybrid BM25 + Semantic Search | Cross-Encoder Reranking | 3 Analysis Modes | Azure-Ready · One Env Var | 22 Evaluation Metrics | 122 Tests · 0 Failures

Prepared: April 2026 · Branch: main · Version 2

3	22	122	1
ANALYSIS MODES	EVAL METRICS	TESTS · 0 FAILURES	ENV VAR TO AZURE

Table of Contents

1. Problem Statement & What the System Solves p.2
2. System Architecture Overview p.2
3. Three Analysis Modes p.3
4. Data Ingestion Pipeline (ETL) p.4
5. Retrieval Layer — What's New p.5
5.1 Classic Dense Retrieval (baseline) p.5
5.2 Hybrid BM25 + Dense + RRF NEW p.5
5.3 Cross-Encoder Reranking NEW p.6

5.4 MMR Diversity Filter NEW p.6

6. LangGraph Agent Pipeline p.7

7. Tech Stack & Azure Production Swap p.7

8. API Reference p.9

9. Frontend Architecture p.9

10. Evaluation Framework — 7 Categories, 22 Metrics NEW p.10

11. How to Run & Evaluate p.13

12. Production Roadmap p.13

13. File & Folder Structure p.14

1. Problem Statement & What the System Solves

Riverty's legal team reviews hundreds of contracts annually. The current process is semi-manual — reviewers open each PDF individually, search for specific clauses, and record findings in spreadsheets. This creates three compounding problems:

TIME COST

Reading full PDFs clause by clause takes hours per contract. Business decisions are delayed waiting for legal sign-off.

INCONSISTENCY

Different reviewers focus on different clauses. No consistent checklist means compliance risks slip through undetected.

NO CROSS-DOCUMENT SEARCH

"Which contracts have unlimited liability?" requires opening every file. There is no way to query the full portfolio simultaneously.

The brief: "Our legal colleagues regularly need to compare contracts and identify contracts that do not contain a certain text passage or contracts in which a company name needs to be updated. So far, this is semi-manual work."

What the System Delivers

Capability	How	Example query
Instant clause search	Hybrid semantic + keyword search across all stored contracts	"Which contracts are missing a GDPR clause?"
Draft comparison	Upload new contract, compare against entire knowledge base	"How does this draft differ from our existing agreements?"
Compliance check	Structured GDPR, termination, liability, jurisdiction check	Returns: compliant: true/false + violations list
Entity name search	BM25 exact-term matching for company names, article numbers	"Which contracts still reference the old company name?"
Source citations	Every answer cites document, page number, and chunk index	Legal team can verify each claim against the original PDF

2. System Architecture Overview

```
USER BROWSER — React + TypeScript SPA
┌──────────────────────────────────────────────────────────────┐
│  [ Analyze Document ]  [ Compare with DB ]  [ Search KB ]    │
│      MODE 1                  MODE 2              MODE 3       │
└──────────────┬──────────────────┬──────────────────┬─────────┘
               │  SSE stream      │  SSE stream      │  SSE stream
               ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────────┐
│                    FastAPI Backend  :8000                     │
│  POST /api/analyze   POST /api/analyze   POST /api/query      │
│  POST /api/compliance                    POST /api/eval/retrieve│
├──────────────────────────┬───────────────────────────────────┤
│  ETL Pipeline            │  RAG Layer                        │
│  PDF / OCR               │  EmbeddingService (BAAI/bge-m3)   │
│  → TextCleaner           │  HybridRetriever  ← BM25 + Dense  │
│  → ContentTypeDetector   │  CrossEncoder Reranker            │
│  → DocumentChunker       │  MMR Diversity Filter             │
│  → EmbeddingService      │  LangGraph Agent (4 nodes)        │
│  → ChromaDB              │                                   │
└──────────────────────────┴───────────────────────────────────┘
         │
         ▼
ChromaDB local disk (demo)  ←──────────→  Azure AI Search (production)
```

The system has two completely independent pipelines that never interfere: the **ETL pipeline** runs once per document at ingest time; the **RAG pipeline** runs at query time. The LLM never sees a contract it does not need — it only sees the specific chunks retrieved for the current question, preventing hallucination by construction.

3. Three Analysis Modes

MODE 1 — Analyse Single Document

Upload any contract and ask questions about it. The document is extracted, analysed, and then **immediately deleted** — it is never added to the knowledge base. Useful for sensitive drafts not yet ready to store.

Also supports structured **compliance check**: returns a JSON result with compliant flag, violations list, and explanation.

`POST /api/analyze · mode=single`

`POST /api/compliance`

MODE 2 — Compare Against Knowledge Base

Upload a draft contract and compare it against all stored contracts. The system retrieves the most similar stored contracts semantically, then presents GPT-4o with both the uploaded document and retrieved contracts for a side-by-side comparison.

Identifies **missing clauses, deviations, and company name mismatches**.

`POST /api/analyze · mode=compare`

MODE 3 — Cross-Contract Query

Ask any question across all stored contracts simultaneously — no file upload needed. Uses the full hybrid retrieval pipeline (BM25 + semantic + reranking). The LangGraph agent classifies the query type and builds the appropriate context.

Answers include **per-document verdicts** for "which contracts" queries.

`POST /api/query`

Privacy guarantee — Mode 1 & 2: Temporary files are deleted in a `finally` block in the route handler — this is enforced at the server level, not just the UI label. Temporary documents *never* touch ChromaDB or any persistent storage.

Compliance Check (Mode 1)

A dedicated endpoint performs a structured compliance assessment against default legal guidelines. Returns machine-readable JSON, not a prose stream:

```
POST /api/compliance → ComplianceResult
{
  "compliant": false,
  "violations": [
    "No GDPR / data protection clause found",
    "Termination notice period not specified"
  ],
  "explanation": "The contract does not address data processing obligations..."
}
```

Default checks: GDPR / data protection clause · Termination rights and notice period · Governing law and jurisdiction · Liability cap and limitation clauses · Clear party identification

4. Data Ingestion Pipeline (ETL)

1

Extract

PDFExtractor (PyMuPDF): page-by-page text extraction, handles mixed English/German content. Falls back automatically to **OCRExtractor** (Tesseract 5 with German language model) if text extraction yields empty pages or for `.jpg/.png` files.

2

Transform

TextCleaner: normalises whitespace, pipe characters, and German umlauts. **LanguageDetector**: tags each chunk as `en` or `de`. **ContentTypeDetector**: classifies as `qa` / `legal` / `narrative` to select the appropriate chunking strategy.

Three chunking strategies: (1) *QA*: split at `Q:` markers — one chunk per question/answer pair. (2) *Legal*: split at §, Article N, Clause N — one chunk per clause (atomic unit of legal meaning). (3) *Narrative*: `RecursiveCharacterTextSplitter` at 1500 chars / 200-char overlap.

3

Embed & Load

EmbeddingService: generates 1024-dimensional vectors using `BAAI/bge-m3` (multilingual, runs locally — no API call during ingest). Each chunk is stored in ChromaDB with **14 metadata fields**: source file, page number, chunk index, total chunks, language, content type, MD5 checksum, file size, extraction method, is_scanned flag, embedding model, upload timestamp, text preview, chunking strategy.

4

Deduplication Guard

`ingestion_registry.json` tracks every ingested file by MD5 checksum. Uploading the same file twice returns `status="skipped"` instantly — no re-embedding. Attempting to ingest with a different embedding model returns `409 Conflict` to prevent incompatible vector dimensions from mixing in the same collection.

5. Retrieval Layer — What's New

5.1 Classic Dense Retrieval (baseline)

The original retrieval pipeline used cosine similarity on `BAAI/bge-m3` embeddings: the query is embedded to a 1024-dim vector, ChromaDB returns the top-K most similar chunks, and a threshold of ≥0.40 filters out low-relevance results. This works well for paraphrased queries but misses exact legal terms.

5.2 Hybrid BM25 + Dense + Reciprocal Rank Fusion NEW

> **Why hybrid matters:** Dense vectors handle paraphrased queries ("ending the agreement" ↔ "termination"). BM25 handles exact legal terms that dense search can miss — entity names like "Riverty GmbH", article numbers like "§12", regulation citations like "Article 17 GDPR". RRF merges both ranked lists without requiring calibrated score scales.

```
# ============================================================ # DEMO MODE: BM25 index built in-memory over ChromaDB chunk texts. # PRODUCTION SWAP → Azure AI Search hybrid mode: # Azure AI Search runs BM25 + vector natively in a single API call. # No separate BM25 index needed — remove hybrid_retriever.py entirely. # ============================================================
```

The `HybridRetriever` class maintains an in-memory `BM25Okapi` index over all ChromaDB chunks. The index is built lazily on first query and automatically refreshed when new documents are ingested.

Reciprocal Rank Fusion (RRF) merges the two ranked lists using the formula:

```
RRF_score(chunk C) = Σ  1 / (K + rank_of_C_in_list_L)
                    L ∈ {dense, BM25}

K = 60 (standard constant — reduces impact of extreme top ranks)

Example: chunk "§12 Termination" ranked 1st in BM25 and 3rd in dense:
  RRF = 1/(60+1) + 1/(60+3) = 0.01639 + 0.01587 = 0.03226

Chunk appearing in only one list always scores lower than one appearing in both.
Fallback: if BM25 returns nothing → dense only; if dense returns nothing → BM25 only.
```

5.3 Cross-Encoder Reranking NEW

After RRF merging, the top candidates are reranked using `cross-encoder/ms-marco-MiniLM-L-6-v2`:

Bi-encoder (retrieval)	Cross-encoder (reranking)
Encodes query and chunk independently, then compares embeddings. Fast — works at scale.	Reads query and chunk text jointly in one forward pass. Much more accurate relevance signal.
May confuse "payment terms agreement" with "termination agreement" due to shared context.	Correctly distinguishes fine-grained relevance — critical for short legal clauses.

The cross-encoder model (~80 MB) is lazy-loaded on first query and cached. If unavailable (e.g. no internet on first run), the system falls back silently to sorting by cosine similarity — no error, retrieval continues to work.

5.4 MMR Diversity Filter NEW

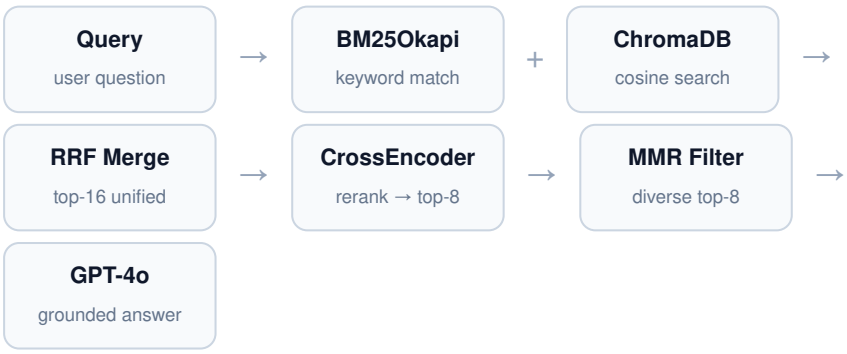
After reranking, Maximal Marginal Relevance (MMR) selects a diverse subset of top-K chunks. Without it, the top 8 chunks might all be slight variations of the same termination clause from the same document — wasting the LLM context window.

```
MMR_score(candidate C) = λ × relevance(C)  −  (1−λ) × max_similarity(C, already_selected)

λ = 0.6 → 60% relevance weight / 40% diversity penalty
Inter-chunk similarity: Jaccard overlap of token sets (no extra model needed)

Result: diverse set of chunks from different contracts and different clause types
        → GPT-4o receives broader evidence → better answer quality
```

Complete Retrieval Pipeline (Mode 3)



6. LangGraph Agent Pipeline

The query pipeline is implemented as a 4-node LangGraph state machine. Each node receives the typed state dict and returns an updated version — independently testable and replaceable.

NODE 1 · QUERY_ROUTER

Classifies the question: `find_clause`, `compare_contracts`, `summarise`, `general_question`, `find_missing`. Determines context-building strategy in Node 3.

NODE 2 · RETRIEVER

Runs HybridRetriever → CrossEncoder → MMR. Returns top-8 diverse, reranked chunks with full metadata (source file, page, chunk index, similarity score).

NODE 3 · REASONER

For `find_missing`: groups chunks by document → per-document context → per-document verdict. Otherwise: flat context string → grounded-answer prompt → GPT-4o streaming.

NODE 4 · FORMATTER

Appends source citations with relevance dots: ● = high relevance (≥0.65), ◖ = relevant (0.40–0.65). Every claim is traceable to a specific document and page.

> **Why LangGraph over a simple chain:** The query router means different query types get different context-building strategies without a monolithic prompt. Each node is independently unit-tested. Future additions (hallucination check node, feedback loop node) can be inserted without refactoring the existing graph.

7. Tech Stack & Azure Production Swap

Backend

Layer	Demo (Today)	Production (Azure)	Why this choice
Vector store	ChromaDB 1.5.5 — zero-config, in-process, local disk	Azure AI Search — hybrid vector + BM25, RBAC, SOC 2	One-line swap; Azure adds enterprise scale and keyword search natively
BM25 index	rank_bm25 — in-memory BM25Okapi over ChromaDB corpus	*Not needed* — Azure AI Search has BM25 built-in	Bridges the gap between ChromaDB (vector only) and Azure AI Search (hybrid)
LLM	AsyncOpenAI GPT-4o (api.openai.com)	AsyncAzureOpenAI GPT-4o — same model, data stays in Azure tenant	Identical SDK; switching is changing one URL + credentials
Embeddings	BAAI/bge-m3 — local, free, offline, 1024-dim, multilingual	Azure OpenAI text-embedding-3-large — 3072-dim, within Azure tenant	bge-m3 runs offline — contract text never leaves the machine during demo
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2 — local ~80 MB	Azure Cognitive Search semantic reranker	Local model sufficient for demo; Azure reranker scales without CPU load
PDF extraction	PyMuPDF 1.27 + pytesseract 0.3.13 (Tesseract 5, German model)	Azure Document Intelligence	Handles handwriting, low-quality scans, table extraction in production
File storage	Local filesystem `./uploads/`	Azure Blob Storage — versioning + legal hold	Versioned contract copies + right-to-erasure support for GDPR
Agent framework	LangGraph 1.1.3	LangGraph (unchanged)	Works natively with Azure OpenAI — no changes needed
Auth	None (single-user local demo)	Azure Active Directory — SSO with existing Riverty credentials	Legal team uses existing login; RBAC per role (viewer / ingester / admin)

Frontend

Tool	What it does	Why chosen
React 18 + TypeScript	UI framework	TypeScript discriminated union `AnalysisMode = 'search' | 'analyze' | 'compare'` makes it impossible to pass the wrong mode to the backend at compile time
Vite	Build tool + dev server	Builds in under 500ms; proxy config eliminates CORS issues during development
TailwindCSS	Styling	Utility-first; no CSS files; consistent design tokens across all components

The Swap Pattern — One Environment Variable

```
APP_ENV=development # → demo stack (ChromaDB, bge-m3 local, OpenAI direct) APP_ENV=production # → Azure stack (Azure AI Search, text-embedding-3-large, Azure OpenAI)
```

All swappable components implement abstract base classes (`BaseLoader`, `BaseExtractor`, `BaseStorage`). The calling code in `pipeline.py` and `agent.py` never imports the concrete class directly — `APP_ENV` selects the right implementation at construction time. No business logic changes.

Six credentials needed for production: `AZURE_OPENAI_API_KEY` · `AZURE_SEARCH_KEY` · `AZURE_BLOB_CONN_STR` · `AZURE_DOC_INTEL_KEY` · `AZURE_TENANT_ID` · `AZURE_CLIENT_ID`

8. API Reference

Method	Endpoint	Request	Response	Mode
GET	`/health`	—	`{status, mode, document_count, version}`	—
POST	`/api/ingest`	`multipart: file`	`{status, chunks_created, language}`	—
POST	`/api/ingest-all`	—	`{results: [...]}`	—
POST	`/api/query`	`{"question": str}`	SSE token stream	3
POST	`/api/analyze`	`multipart: file, question, mode`	SSE token stream	1 / 2
POST	`/api/compliance`	`multipart: file, guidelines?`	`ComplianceResult` JSON	1
GET	`/api/files/{filename}`	—	PDF bytes	—
GET	`/api/suggested-questions`	—	`[{id, text, category}]`	—
POST	`/api/eval/retrieve` NEW	`{"question": str, "top_k": int}`	`{question, chunks[], total_retrieved}`	Eval
GET	`/docs`	—	Swagger UI (interactive)	—

All answer endpoints use **Server-Sent Events (SSE)**: tokens stream as `data: token\n\n`, terminated by `data: [DONE]\n\n`. SSE is preferred over WebSocket because it traverses corporate firewalls as plain HTTP with no protocol upgrade required.

The `/api/eval/retrieve` endpoint is for the evaluation harness only — it runs the full hybrid retrieval pipeline and returns raw chunks without calling GPT-4o, enabling cheap IR metric computation.

9. Frontend Architecture

LAYOUT

Left sidebar: Knowledge base management — drag-drop PDF upload (persistent, stored in ChromaDB), list of all ingested files.

Right panel: Three-tab mode switcher (Search / Analyse / Compare) with mode-specific upload zones, query input, suggested questions, compliance check button, and streaming response display.

KEY DESIGN DECISIONS

- **Temporary uploads** show a blue "Temporary" badge — enforced at server level.
- **Mode switching** clears all response state — no stale content bleed between modes.
- **useDocumentAnalysis** hook encapsulates all temp file + streaming + compliance state.
- **SSE** via `fetch()` + `response.body.getReader()` (not `EventSource`) — allows POST with body.

10. Evaluation Framework — 7 Categories, 22 Metrics NEW

Design principle: Generic NLP metrics (BLEU, ROUGE) measure word overlap. A system can score well on BLEU and still fail to identify a missing GDPR clause. The 22 metrics here are designed specifically for RAG systems operating in the legal domain — each category catches a different class of failure.

Overview — All 22 Metrics at a Glance

1 · Retrieval (3 metrics)

- **Precision@K** — fraction of retrieved chunks that are relevant
- **Recall@K** — fraction of relevant chunks that were retrieved
- **Mean Reciprocal Rank (MRR)** — average of 1/rank of first relevant result

2 · Generation (4 metrics)

- **Faithfulness** — answer stays grounded in retrieved chunks (no hallucination)
- **Answer Relevance** — answer actually addresses the question asked
- **Completeness** — answer covers all expected points from ground truth
- **Hallucination Rate** — fraction of claims unsupported by context

3 · Legal Domain (4 metrics)

- **Contract Hit (C-Hit)** — Mode 2: does the answer reference the correct contract?
- **Keyword Hit (Kw-Hit)** — expected legal keywords present in answer?
- **Absent Keyword Pass (AbsKw)** — system correctly says clause is absent?
- **Comparison Hit (Cmp-Hit)** — comparison answer covers both contracts?

5 · Attribution (2 metrics)

- **Citation Accuracy** — cited source files actually exist in the knowledge base?
- **Citation Coverage** — fraction of answer sentences that have at least one citation?

6 · Robustness (2 metrics)

- **Query Consistency** — consistent answers across paraphrased versions of the same question?
- **Noise Sensitivity** — answer stability when irrelevant noise chunks are injected?

7 · System Performance (2 metrics)

- **Latency** — wall-clock time from request to final SSE token (milliseconds)
- **Cost per Query** — estimated OpenAI token cost per query (USD)

4 · Context Quality (2 metrics)

- **Context Precision** — are retrieved chunks actually useful to GPT-4o?
- **Context Recall** — were any relevant chunks missed during retrieval?

Implementation Architecture — Pluggable Metrics Package

```
backend/tests/eval/
├── run_eval.py                 # Orchestrator — runs all 3 modes, collects all 22 metrics
├── ground_truth.json           # Expected answers and keywords per test question
└── metrics/                    # Pluggable metric modules
    ├── answer_metrics.py       # Faithfulness, Answer Relevance, Completeness, Hallucination
    ├── context_metrics.py      # Context Precision, Context Recall
    ├── attribution_metrics.py  # Citation Accuracy, Citation Coverage
    ├── cost_metrics.py         # Cost per Query, Session Cost aggregation
    └── robustness_metrics.py   # Query Consistency, Noise Sensitivity + paraphrase generator
```

Each module is independently importable and testable. New metrics can be added by creating a new module and registering it in `run_eval.py` — no changes to the core pipeline.

Retrieval Metrics (Category 1) — Using the Eval Endpoint

IR metrics are computed via `POST /api/eval/retrieve` which runs HybridRetriever → CrossEncoder → MMR and returns raw chunks. This evaluates real production retrieval quality cheaply — no LLM token costs.

Metric	Formula
Precision@K	|{retrieved} ∩ {relevant}| / K
Recall@K	|{retrieved} ∩ {relevant}| / |{relevant}|
MRR	Mean over queries of 1 / rank(first relevant chunk)

Faithfulness — Dual Backend (automatic fallback)

PRIMARY — LLM JUDGE (GPT-4O-MINI)

GPT-4o-mini receives: the question, retrieved context, and the system's answer. It scores faithfulness 0–10 based on whether every factual claim in the answer appears in the provided context. Score normalised to 0–1. Most accurate method.

Requires: OPENAI_API_KEY

FALLBACK — LOCAL EMBEDDING COSINE

Mean cosine similarity between context chunk embeddings and the answer embedding, computed locally using `BAAI/bge-m3`. Always available offline — no API call required. Lower absolute accuracy but always available.

Automatic fallback if API key absent

Legal Domain Metrics (Category 3) — Why They Matter

These are the most important metrics for Riverty's legal use case. Each maps directly to a real workflow scenario:

Metric	Mode	Real scenario it validates
Contract Hit	2	When comparing a draft, does the system find and reference the correct comparison contracts?
Keyword Hit	1, 2, 3	When asked about GDPR, termination, or liability, does the answer contain the expected legal terms?
Absent Keyword Pass	1	When a clause is genuinely missing from a contract, does the system correctly report "not found" rather than hallucinating a clause?
Comparison Hit	3	For "which contracts differ in X", does the answer correctly identify all relevant contracts — not just the most similar one?

Robustness Testing

Query Consistency: The `generate_paraphrases()` function creates semantically-equivalent variants of each test question. The system must give consistent answers regardless of phrasing — critical because different legal reviewers may phrase the same question differently.

Noise Sensitivity: The `inject_noise_chunks()` function appends irrelevant distractor chunks to the retrieved context. A robust system should maintain its answer quality; a fragile one will be thrown off by irrelevant material. Low noise sensitivity score = robust.

Evaluation Output Sample

```
# results/eval_20260402_143022.json  (generated by run_eval.py)
{
  "run_timestamp": "2026-04-02T14:30:22",
  "model": "gpt-4o",
  "embedding_model": "BAAI/bge-m3",
  "top_k": 8,
  "summary": {
    "mode_1": {
      "contract_hit": 0.92,  "keyword_hit": 0.88,
      "faithfulness": 0.91,  "hallucination_rate": 0.04,
      "avg_latency_s": 2.3,  "avg_cost_usd": 0.004
    },
    "mode_2": {
      "precision_at_k": 0.75,  "recall_at_k": 0.68,  "mrr": 0.82,
      "contract_hit": 0.85,    "faithfulness": 0.89,
      "context_precision": 0.81,  "citation_accuracy": 0.97
    },
    "mode_3": {
      "keyword_hit": 0.86,  "comparison_hit": 0.80,
      "query_consistency": 0.84,  "noise_sensitivity": 0.12,
      "citation_coverage": 0.93,  "avg_latency_s": 3.1
    }
  }
}
```

11. How to Run & Evaluate

Backend Setup (5 minutes)

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Set OPENAI_API_KEY in .env

uvicorn app.main:app --reload --port 8000
# → http://localhost:8000/docs
```

Frontend Setup

```
cd frontend
npm install && npm run dev
# → http://localhost:5173
```

Ingest Contracts

```
# Via UI: drag PDF to left sidebar
# Via API:
curl -X POST http://localhost:8000/api/ingest \
     -F "file=@contract.pdf"

# Batch ingest:
bash backend/run_pipeline.sh --ingest-only
```

Run Tests

```
cd backend
bash run_tests.sh
# → 122 tests, 0 failed

# Run evaluation harness:
python tests/eval/run_eval.py \
  --modes 1,2,3 --top-k 8 \
  --output-dir results/
```

Note on the evaluation harness: The backend server must be running before executing `run_eval.py`. Faithfulness scoring uses GPT-4o-mini (requires `OPENAI_API_KEY`). If the key is absent, the harness automatically switches to the local embedding fallback — all other metrics still compute normally.

12. Production Roadmap

✓ READY TODAY — V2

- Full ETL pipeline (PDF + OCR, German + English)
- 3 analysis modes with SSE streaming
- Hybrid BM25 + dense retrieval + RRF
- Cross-encoder reranking + MMR diversity filter
- Compliance check — structured JSON result
- Evaluation harness: 22 metrics, 7 categories
- React frontend — drag-drop, citations, 3 modes
- 122 tests, 7 Architecture Decision Records
- Azure swap pattern — every component documented

SHORT-TERM (1–2 WEEKS)

- `GET /api/files` listing — sidebar persists on page refresh
- Re-index ChromaDB with consistent 1500-char chunks
- Stream compliance check response (currently waits full result)
- VITE_ prefix for BASE_URL in frontend .env

PRODUCTION SWITCH

- Set `APP_ENV=production` + 6 Azure credentials in `.env`
- Deploy to Azure Container Apps (Dockerfile included)
- Azure AD SSO — legal team uses existing Riverty login
- CORS update for production domain
- Terraform modules for Azure resources (~2–3 days)
- Remove `hybrid_retriever.py` — Azure AI Search handles BM25 natively

ADVANCED ENHANCEMENTS

- Redis cache for embeddings + frequent queries
- Azure Service Bus for async ingestion at scale
- SharePoint integration — auto-ingest on upload
- Contract type classification (NDA / Service / Vendor)
- Multi-file batch upload (zip ingest)
- RAGAS integration for automated faithfulness scoring
- Audit log for every query (compliance trail)

Adoption Plan

PHASE 1 — PILOT

2–3 legal reviewers, 2 weeks. Pre-load 50 most-referenced contracts. Measure time-to-answer before vs. after. Collect feedback on missed clauses.

PHASE 2 — ITERATE

Tune similarity threshold based on real feedback. Add SharePoint integration. Build query templates for the 5 most common review tasks.

PHASE 3 — FULL ROLLOUT

Azure AD SSO — existing Riverty credentials. Role-based access: viewers / ingesters / admins. Audit log for every query.

Important — what the system does NOT do: It does not give legal advice. It does not replace lawyers — it surfaces evidence so humans can decide faster and more consistently. It does not interpret contract validity. Every answer cites its source — every claim is auditable.

13. File & Folder Structure

```
riverty-contract-review/
├── CLAUDE.md                          # Coding conventions
├── backend/
│   ├── requirements.txt               # All Python dependencies pinned
│   ├── .env.example                   # Template for all environment variables
│   ├── Dockerfile                     # Python 3.12 production image
│   ├── run_tests.sh                   # pytest + coverage (122 tests)
│   └── app/
│       ├── main.py                    # FastAPI app + router registration
│       ├── config.py                  # Pydantic Settings — single .env read
│       ├── models.py                  # ComplianceResult + shared Pydantic models
│       ├── api/routes/
│       │   ├── query.py               # POST /api/query  (MODE 3 — SSE)
│       │   ├── analyze.py             # POST /api/analyze (MODE 1 & 2 — SSE)
│       │   ├── compliance.py          # POST /api/compliance (JSON)
│       │   ├── ingest.py              # POST /api/ingest
│       │   ├── eval_retrieve.py       # POST /api/eval/retrieve [NEW — eval only]
│       │   ├── files.py               # GET /api/files/{filename}
│       │   ├── suggestions.py         # GET /api/suggested-questions
│       │   └── health.py              # GET /health
│       ├── etl/
│       │   ├── pipeline.py            # IngestionPipeline orchestrator
│       │   ├── registry.py            # MD5 dedup + embedding model guard
│       │   ├── extractors/            # PDFExtractor, OCRExtractor (Strategy Pattern)
│       │   ├── transformers/          # TextCleaner, LanguageDetector, Chunker
│       │   └── loaders/               # ChromaLoader (demo) ↔ AzureSearchLoader (prod)
│       ├── rag/
│       │   ├── embeddings.py          # EmbeddingService (bge-m3 / Azure swap)
│       │   ├── retriever.py           # Classic cosine retriever (compliance/analyze)
│       │   ├── hybrid_retriever.py    # BM25 + dense + RRF [NEW]
│       │   ├── reranker.py            # CrossEncoder + MMR filter [NEW]
│       │   ├── agent.py               # LangGraph 4-node graph
│       │   ├── llm_client.py          # OpenAI / AzureOpenAI factory
│       │   ├── document_analyzer.py   # MODE 1 & 2 analysis functions
│       │   └── document_grouper.py    # Per-document grouping for MODE 3
│       └── storage/
│           ├── local_storage.py       # Local ./uploads/ (demo)
│           └── azure_blob_storage.py  # Azure Blob Storage (production)
│
├── frontend/src/
│   ├── App.tsx                        # Root — three-mode layout
│   ├── services/                      # api.ts, streaming.ts (SSE helpers)
│   ├── hooks/useDocumentAnalysis.ts   # File + streaming + compliance state
│   └── components/                    # ModeSelector, FileUpload, StreamingResponse, etc.
│
├── docs/
│   ├── decisions.md                   # ADR-001 to ADR-007
│   ├── implementation-status.md
│   └── setup.md
│
└── backend/tests/
    ├── unit/                          # 122 fast offline tests (mocked externals)
    └── eval/                          # [NEW] evaluation harness
        ├── run_eval.py                # Main script — all 3 modes, 22 metrics
        ├── ground_truth.json
        └── metrics/                   # answer, context, attribution, cost, robustness
```

Riverty Contract Review · AI-Powered Legal Analysis System · v2 · April 2026 · Branch: `main`
Built with FastAPI · LangGraph · BAAI/bge-m3 · rank_bm25 · ChromaDB · React 18 · Azure-Ready